

Moko Payout API

Guide d'intégration marchand

Version 1.0 — Juin 2026

Sommaire

PARTIE I — Découverte avec Postman

1. Présentation de l'API et pré-requis
2. Comprendre les 4 identifiants clés
3. Importer et configurer la collection Postman
4. Exécuter le flux complet pas à pas

PARTIE II — Intégration en production

5. Architecture recommandée
6. Stockage et rotation des clés
7. Signature HMAC — spécification complète
8. Référence des endpoints
9. Les 4 identifiants dans votre code de production
10. Webhooks — réception et vérification
11. Gestion des erreurs et stratégie de retry
12. Idempotency en pratique
13. Exemples bout-en-bout (Python et Node.js)
14. Checklist de mise en production
15. Support

PARTIE I — Découverte avec Postman

Cette première partie est destinée à valider rapidement votre intégration et tester les endpoints sans écrire de code. La collection Postman fournie gère automatiquement la signature HMAC pour vous.

1. Présentation de l'API et pré-requis

L'API Moko Payout permet de déclencher, par programmation, des transferts Mobile Money depuis votre portefeuille Moko vers des bénéficiaires pré-enregistrés. Elle est destinée aux intégrations machine-à-machine (serveur à serveur) et remplace, pour ce cas d'usage, l'utilisation du tableau de bord marchand.

Cas d'usage typiques

- Plateforme éducative : versement automatique des revenus aux enseignants à la fin de chaque cours.
- Marketplace : reversement aux vendeurs après livraison validée.
- Gestion RH : paiement de salaires ou commissions à des employés et freelances.
- Programmes de cashback : redistribution de fidélité aux clients.

Pré-requis

- Compte marchand Moko actif (avec votre `merchant_code`).
- Paire de clés API (`public_key` + `secret_key`) émise par le support Moko.
- Postman version 10 ou supérieure pour la partie découverte.
- Un environnement Python, Node.js, PHP, Go, Java ou autre, capable de calculer HMAC-SHA256, pour la partie intégration.

Gestion des clés API

Votre `secret_key` vous est communiquée une seule fois lors de son émission. Si vous la perdez, il faut révoquer la clé et en émettre une nouvelle. Conservez-la dans un coffre-fort numérique (1Password, Bitwarden, AWS Secrets Manager, HashiCorp Vault...) et ne la committez jamais dans votre dépôt git.

2. Comprendre les 4 identifiants clés

Avant de manipuler la collection Postman et avant d'écrire la moindre ligne de code de production, il est essentiel de bien comprendre les quatre identifiants utilisés par l'API. Deux sont fournis par vous (le marchand), deux sont fournis par Moko. Confondre leur rôle est la source d'erreurs n°1 chez les nouveaux intégrateurs.

2.1 Vue d'ensemble

Identifiant	Qui le fournit	Quand	Format	Rôle principal
merchant_recipient_id	Vous (marchand)	Création du bénéficiaire	Chaîne libre ≤128 chars	Votre ID interne du bénéficiaire
recipient_id	Moko	Réponse de Create Recipient	rec_<32 hex>	ID Moko du bénéficiaire (à passer dans les payouts)
merchant_reference	Vous (marchand)	Initiation d'un payout	Chaîne libre ≤128 chars	Votre clé d'idempotency
payout_id	Moko	Réponse de Create Payout	PAY-YYYYMMDDhhmmss-XXXXXX	ID Moko du payout (= reference)
kyc_reference	Vous (marchand)	Création du bénéficiaire	Chaîne libre ≤128 chars	Référence KYC interne (audit)

Note : nous avons listé cinq lignes ci-dessus mais nous parlons des « 4 identifiants clés ». En effet, `recipient_id` est la contrepartie directe de `merchant_recipient_id` (le couple Moko/marchand pour la même entité bénéficiaire). Détaillons maintenant chacun en profondeur.

2.2 `merchant_recipient_id` — votre ID interne du bénéficiaire

C'est l'identifiant que VOUS choisissez pour désigner un bénéficiaire dans VOTRE système. Vous décidez de sa valeur, de sa structure, et de sa signification. Moko le stocke tel quel et vous le restitue dans toutes les réponses.

Format et contraintes

- Chaîne de caractères libre, longueur maximale 128 caractères.
- Doit être unique parmi VOS bénéficiaires (deux marchands différents peuvent utiliser la même valeur, pas de conflit).
- Caractères recommandés : alphanumérique, tirets, underscores. Évitez les espaces et caractères spéciaux pour faciliter le debug.

Côté Postman

Pré-rempli dans les variables de la collection avec la valeur `teacher_42`. Vous pouvez le changer dans l'onglet Variables avant d'envoyer Create Recipient. La même valeur est automatiquement injectée dans le body de la requête grâce à la syntaxe `{{merchant_recipient_id}}`.

Côté production

Utilisez la clé primaire ou l'identifiant métier de votre table de bénéficiaires interne. Exemples :

- Plateforme éducative : teacher_<id_enseignant>, par exemple teacher_4521.
- Marketplace : vendor_<id_vendeur>, par exemple vendor_a3f8c92b.
- RH : emp_<matricule>, par exemple emp_00231.

Bonnes pratiques

- Utilisez un préfixe qui identifie le type d'entité (teacher_, vendor_, emp_).
- Ne changez jamais la valeur pour un bénéficiaire existant. Si vous restructurez votre DB, archivez l'ancien recipient et créez-en un nouveau.
- Si vous tentez de créer deux fois un bénéficiaire avec le même merchant_recipient_id, vous recevez une erreur 409 recipient_already_exists. Utilisez plutôt Get pour vérifier l'existence et List pour rechercher.

2.3 recipient_id — l'ID Moko du bénéficiaire

C'est l'identifiant que MOKO génère quand vous créez un bénéficiaire. Vous le recevez dans la réponse de Create Recipient et vous l'utilisez ensuite pour référencer ce bénéficiaire dans les payouts et les endpoints GET/PATCH/DELETE.

Format

- Préfixe rec_ suivi de 32 caractères hexadécimaux.
- Exemple : rec_7876fc73e5b580da75d5ec36ce882dce
- Vous ne le générez jamais. Vous ne le devinez jamais.

Côté Postman

La requête Create Recipient comporte un test script qui capture automatiquement le recipient_id de la réponse et le stocke dans la variable de collection portant le même nom. Les requêtes Get, Update, Delete et Create Payout l'utilisent ensuite via {{recipient_id}}, sans que vous ayez à copier-coller quoi que ce soit.

Côté production

À la création, vous DEVEZ stocker recipient_id dans VOTRE table de bénéficiaires, à côté de votre clé primaire. Schéma type :

```
CREATE TABLE beneficiaries (  
  id          BIGINT PRIMARY KEY,          -- votre clé primaire  
  full_name   VARCHAR(255),  
  phone       VARCHAR(32),  
  operator    VARCHAR(16),  
  moko_recipient_id VARCHAR(64) NOT NULL,  -- le rec... renvoyé par Moko  
  moko_status VARCHAR(32),                 -- ACTIVE, PENDING_KYC, etc.  
  created_at  DATETIME,  
  UNIQUE KEY uq_moko_recipient (moko_recipient_id)  
);
```

Bonnes pratiques

- Stockez `recipient_id` immédiatement après la création. Si vous le perdez, vous devrez interroger Moko via `Get /v1/recipients` (paginé) pour le retrouver, ce qui est coûteux.
- Ne tentez jamais de le calculer vous-même. Le format est susceptible d'évoluer.
- Utilisez-le partout dans votre code interne pour référencer les bénéficiaires côté Moko. Conservez votre propre clé primaire pour vos liens internes (factures, etc.).

2.4 `merchant_reference` — votre clé d'idempotency

C'est l'identifiant que VOUS choisissez pour CHAQUE intention de payout. Il joue le rôle critique de clé d'idempotency : si vous renvoyez la même requête deux fois avec la même valeur, Moko détecte le doublon et NE refait PAS le payout. C'est la protection ultime contre les doubles prélèvements en cas de panne réseau ou serveur.

Pourquoi c'est critique

Imaginez : votre serveur envoie un `Create Payout` à Moko. Le paiement est exécuté côté Moko, mais votre serveur plante avant de recevoir la réponse 202. Quand votre serveur redémarre, comment éviter de relancer le paiement et d'envoyer 100 CDF en double au bénéficiaire ? Réponse : vous renvoyez la même requête avec le MÊME `merchant_reference`. Moko renvoie 200 avec le payout existant. Aucun double paiement.

Format et contraintes

- Chaîne de caractères libre, longueur maximale 128 caractères.
- Unique parmi VOS payouts API. Réutilisable plus tard (>24h) mais à éviter.
- Insensible à la casse côté unicité — préférez tout minuscule ou tout majuscule.

Côté Postman

Pré-rempli avec la valeur dynamique `INV-{{$timestamp}}`. Postman remplace `{{$timestamp}}` par le timestamp Unix actuel à CHAQUE clic sur Send. Chaque envoi génère donc un payout différent, pratique pour le test.

Pour tester le mécanisme d'idempotency : hard-codez une valeur fixe comme `INV-TEST-001` dans le body, envoyez deux fois. Vous verrez :

- 1er envoi : 202 Accepted avec un nouveau payout.
- 2e envoi : 200 OK avec le MÊME payout. Aucun nouveau prélèvement.

Côté production — c'est ici que c'est crucial

Il existe DEUX approches recommandées pour générer `merchant_reference`. Choisissez celle qui correspond à votre architecture, mais respectez les règles d'or ci-dessous.

Approche A — Référence métier déterministe

Utilisez votre propre identifiant métier qui désigne de manière unique et déterministe l'intention de payout.

Exemples :

- Salaire mensuel : salary-2026-06-emp42 (année-mois-employé).
- Commission de vente : commission-order-12345 (lié à votre id de commande).
- Reversement enseignant : payout-course-789-teacher-42 (cours + enseignant).

Avantage : lisible dans les logs, déterministe (même intention = même ref), impossible à dupliquer accidentellement. Si vous relancez physiquement le batch de paie de juin, vous obtiendrez naturellement les mêmes refs, donc 200 OK sur les payouts déjà passés.

Approche B — UUID stocké en amont

Étape par étape :

- Avant d'appeler Moko, générez un UUID v4 dans votre code.
- Stockez-le dans votre table de payouts EN BASE, avec un statut initial pending_send.
- Appelez Moko en passant cet UUID comme merchant_reference.
- Mettez à jour votre table après la réponse Moko.
- Si votre serveur plante entre étape 2 et étape 4, votre cron de récupération relance l'appel Moko avec le MÊME UUID en lisant votre table.

Règles d'or

- Stockez ou rendez déterministe AVANT l'appel HTTP. Jamais après.
- Loggez merchant_reference systématiquement, c'est la clé qui relie vos logs aux logs Moko.
- Pour les retries automatiques (erreur 5xx, timeout), réutilisez la même valeur.

Anti-pattern absolu

Ne faites JAMAIS ceci : `merchant_reference = str(uuid.uuid4())` juste avant l'appel, sans stockage préalable. Si votre code plante après le `uuid4()` mais avant l'appel, votre retry générera un nouvel UUID et causera un double paiement. Stockage AVANT, toujours.

2.5 payout_id — l'ID Moko du payout

C'est l'identifiant que MOKO génère quand vous créez un payout. Vous le recevez dans la réponse de Create Payout et vous l'utilisez pour appeler Get Payout, et vous le retrouvez dans les webhooks de changement d'état.

Format

- Préfixe PAY- suivi de YYYYMMDDhhmmss (timestamp UTC) + tiret + 6 caractères hex aléatoires.
- Exemple : PAY-20260603135305-D82B5C
- Notez que dans la réponse, le champ payout_id et le champ reference contiennent la MÊME valeur. Nous exposons les deux pour la lisibilité.

Côté Postman

Le test script de Create Payout capture automatiquement payout_id de la réponse et le stocke dans la variable de collection. La requête Get Payout Status l'utilise via {{payout_id}}.

Côté production

À la création, vous DEVEZ stocker payout_id dans VOTRE table de payouts, à côté de votre merchant_reference. Schéma type :

```
CREATE TABLE moko_payouts (  
  id BIGINT PRIMARY KEY, -- votre clé primaire  
  merchant_reference VARCHAR(128) NOT NULL UNIQUE, -- votre clé d'idempotency  
  moko_payout_id VARCHAR(64), -- le PAY-... renvoyé par Moko  
  recipient_id BIGINT, -- FK vers votre table beneficiaries  
  amount DECIMAL(18,2),  
  currency VARCHAR(8),  
  moko_status VARCHAR(32), -- RESERVED, COMPLETED, ...  
  created_at DATETIME,  
  completed_at DATETIME,  
  KEY idx_moko_payout (moko_payout_id),  
  KEY idx_status (moko_status)  
);
```

Utilisation

- Vous l'utilisez pour appeler GET /v1/payouts/{payout_id} et vérifier l'évolution du statut.
- Vous le retrouvez dans le champ data.payout_id de tous les webhooks. C'est par cet ID que vous identifiez à quel payout l'événement se rapporte.
- Vous l'affichez dans vos rapports internes et vos écrans de monitoring.

2.6 kyc_reference — votre référence KYC du bénéficiaire

C'est un identifiant OPTIONNEL que vous fournissez à la création d'un bénéficiaire. Moko ne l'utilise pour aucune logique métier — il sert uniquement à enrichir votre audit et faciliter les réconciliations avec votre propre système KYC.

Format

- Chaîne de caractères libre, longueur maximale 128 caractères, optionnelle.
- Peut être null ou omise dans le body de Create Recipient.

Côté Postman

L'exemple par défaut dans la collection est TEACHER-2026-042. Vous pouvez le modifier directement dans le body de Create Recipient. Si vous ne voulez pas le renseigner, mettez la valeur null.

Côté production

Utilisez l'identifiant interne de votre processus KYC pour ce bénéficiaire. Exemples concrets :

Type de bénéficiaire	Exemple de kyc_reference
Enseignant	TEACHER-CV-2026-042 (votre identifiant de dossier RH)
Vendeur marketplace	VENDOR-KYC-2026-A3F8 (votre identifiant de vérification)
Employé	EMP-CONTRAT-00231 (numéro de contrat de travail)
Freelance	FREELANCE-NIF-1234567 (numéro fiscal du prestataire)
Client cashback	USER-IDCARD-CD-9988 (référence pièce d'identité)

Pourquoi le renseigner

- Audit : en cas de demande d'une autorité de régulation ou d'un audit interne, vous pouvez croiser instantanément vos payouts Moko avec vos dossiers KYC.
- Débogage : si un bénéficiaire vous signale un problème, retrouver la référence dans la console Moko est immédiat.
- Conformité : démontre que vous tracez l'identité du bénéficiaire au moment du payout.

Bonnes pratiques

- Ne mettez JAMAIS le numéro de pièce d'identité brut (NIF, CNI, passeport) en kyc_reference. Mettez une RÉFÉRENCE à votre dossier KYC interne (qui contient lui-même la pièce, chiffrée et auditée chez vous).
- Préfixez par le type de document (TEACHER-, VENDOR-, EMP-...) pour la lisibilité dans la console.
- Évitez les espaces et caractères spéciaux.

kyc_reference vs merchant_recipient_id

Ces deux champs sont distincts et complémentaires. merchant_recipient_id est l'ID stable du bénéficiaire dans VOTRE système (ex. teacher_42). kyc_reference est l'ID du DOSSIER KYC associé à ce bénéficiaire chez vous (ex. TEACHER-CV-2026-042). Le premier ne change jamais ; le second peut évoluer si vous re-validez la KYC du bénéficiaire.

2.7 Récapitulatif visuel du flux des identifiants

VOTRE SYSTÈME	MOKO PAYOUT API
1. CRÉER UN BÉNÉFICIAIRE	
Vous générez : - merchant_recipient_id - kyc_reference (optionnel)	→ Moko reçoit le body Moko enregistre Moko génère : - recipient_id (rec_...) ← Réponse 201 contenant recipient_id
Vous stockez recipient_id dans votre table beneficiaries	
2. INITIER UN PAYOUT	
Vous générez (et stockez) : - merchant_reference Vous fournissez : - recipient_id (du step 1)	→ Moko reçoit le body Moko vérifie idempotency Moko génère : - payout_id (PAY-...) ← Réponse 202 contenant payout_id
Vous stockez payout_id dans votre table moko_payouts	
3. SUIVRE LE STATUT (option A : polling)	
Vous appelez GET /v1/payouts/{{payout_id}}	← Réponse 200 avec status
3. SUIVRE LE STATUT (option B : webhook)	
	→ Moko vous push POST /votre-endpoint avec data.payout_id
Vous retrouvez le payout dans votre table par moko_payout_id, mettez à jour votre statut.	
	← Vous répondez 200 OK

3. Importer et configurer la collection Postman

3.1 Importer le fichier

Dans Postman, cliquez sur Import en haut à gauche et déposez le fichier Moko_Payout_API.postman_collection.json. La collection apparaît dans la barre latérale sous le nom Moko Payout API et contient quatre dossiers : Health, Recipients, Payouts, Balance.

3.2 Renseigner les variables

Cliquez sur le nom de la collection, puis sur l'onglet Variables. Renseignez les variables suivantes :

Variable	Qui la fournit	Valeur
base_url	Moko	https://payouts.gofreshpay.com (déjà rempli)
public_key	Moko (vous communiqué)	Votre clé publique pk_test_... ou pk_live_...
secret_key	Moko (vous communiqué)	Votre clé secrète sk_test_... ou sk_live_...
merchant_recipient_id	Vous	Défaut teacher_42, modifiable
merchant_reference	Vous	Défaut INV-{{\$timestamp}} (généré dynamiquement)
recipient_id	Capturé automatiquement	Laisser vide, rempli après Create Recipient
payout_id	Capturé automatiquement	Laisser vide, rempli après Create Payout

Colonne Current Value

Pour vos clés, remplissez la colonne Current Value uniquement, pas Initial Value. Cela évite que vos secrets soient synchronisés avec Postman Cloud si vous utilisez un workspace partagé.

4. Exécuter le flux complet pas à pas

Suivez les requêtes dans l'ordre. Chaque étape capture automatiquement les identifiants dont la suivante a besoin.

Étape A — Health probe

```
GET https://payouts.gofreshpay.com/v1/health

Réponse 200 OK :
{
  "status": "ok",
  "merchant_code": "mQ685ZR8tik2b",
  "key_id": "key_9f1d1a8145ded7d6fb8ecc41306c254e"
}
```

Étape B — Create recipient

```
POST /v1/recipients
{
  "merchant_recipient_id": "teacher_42",
  "full_name": "Jean MUKENDI",
  "phone": "+243812345678",
  "operator": "mpesa",
  "country": "CD",
  "kyc_reference": "TEACHER-2026-042"
}
```

Réponse 201 contenant recipient_id (rec_...) qui est automatiquement capturé dans la variable recipient_id de la collection.

Étape C — Create payout test 100 CDF

```
POST /v1/payouts
{
  "recipient_id": "{{recipient_id}}",
  "amount": 100,
  "currency": "CDF",
  "merchant_reference": "{{merchant_reference}}",
  "description": "Test 100 CDF",
  "callback_url": null
}
```

Réponse 202 avec payout_id (PAY-...) au statut RESERVED, capturé automatiquement.

Étape D — Suivre le statut

Relancez Get payout status jusqu'à voir le statut passer à COMPLETED.

```
GET /v1/payouts/{{payout_id}}
```

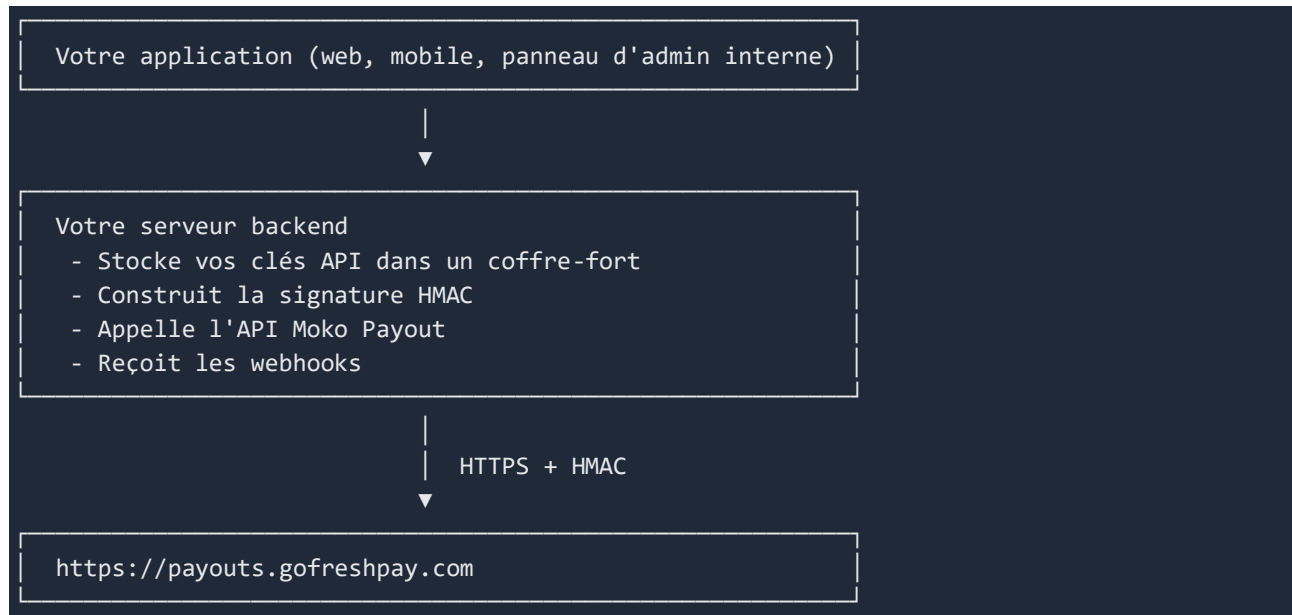
Étape E — Consulter le solde

```
GET /v1/balance
```

PARTIE II — Intégration en production

5. Architecture recommandée

L'API Moko Payout est conçue pour s'intégrer côté serveur de votre système.



Règles d'or

- Les clés API ne doivent jamais quitter votre backend.
- Les appels à l'API Moko sortent uniquement depuis votre serveur, jamais depuis le navigateur de vos utilisateurs.
- L'URL de webhook que vous fournissez doit être un endpoint HTTPS public sur votre serveur, capable de répondre 200 rapidement (< 10 secondes).

6. Stockage et rotation des clés

Bonne pratique	Mauvaise pratique
Variable d'environnement chargée au démarrage	Constante hardcodée dans le code source
Coffre-fort de secrets (AWS Secrets Manager, Vault)	Fichier de config commité dans git
Fichier .env hors git, droits 600	Fichier .env commité dans git
Chiffrement at-rest si stockage en base	Stockage en clair dans une table partagée

La rotation se fait avec une fenêtre de transition de 24h pendant laquelle l'ancien et le nouveau secret sont valides. Demandez la rotation au support, mettez à jour votre coffre-fort, redéployez, l'ancien secret s'invalide automatiquement après 24h.

7. Signature HMAC — spécification complète

7.1 En-têtes obligatoires

En-tête	Valeur
X-Moko-Key	Votre public_key
X-Moko-Timestamp	Millisecondes UTC, tolérance ±60 secondes
X-Moko-Nonce	UUID v4 unique, non réutilisable pendant 5 minutes
X-Moko-Signature	HMAC-SHA256 hexadécimale

7.2 Canonical payload

```
MÉTHODE_HTTP_MAJUSCULES\n/chemin?query_triée_alphabétiquement\nsha256_hex_du_body\ntimestamp_en_millisecondes\nnonce_uuid
```

Détails importants

- Méthode HTTP en majuscules (GET, POST, PATCH, DELETE).
- Chemin inclut /v1/. Query string triée par clé alphabétiquement.
- sha256 du body en hexa minuscule. Body vide = sha256("").
- Timestamp en millisecondes (Date.now() en JS, int(time.time()*1000) en Python).
- Signature = HMAC-SHA256(secret_key, canonical) en hexa.

8. Référence des endpoints

Préfixe : /v1 sur https://payouts.gofreshpay.com. Toutes les requêtes signées HMAC. Toutes les réponses JSON.

8.1 Health

GET

/v1/health

Vérification de connectivité et d'authentification.

8.2 Recipients

POST

/v1/recipients

Enregistrer un nouveau bénéficiaire.

Champ	Type	Obligatoire
merchant_recipient_id	string ≤128	oui
full_name	string 2-255	oui
phone	string +243xxx	oui
operator	mpesa airtel orange afrimoney	oui

country	CD	non, défaut CD
kyc_reference	string ≤128	non

GET /v1/recipients

Lister vos bénéficiaires (paginé : status, page, page_size).

GET /v1/recipients/{recipient_id}

Obtenir les détails d'un bénéficiaire.

PATCH /v1/recipients/{recipient_id}

Modifier un bénéficiaire (cooldown 24h sur champs sensibles).

DELETE /v1/recipients/{recipient_id}

Archiver (soft delete).

8.3 Payouts

POST /v1/payouts

Initier un payout.

Champ	Type	Obligatoire
recipient_id	rec_... (de Create Recipient)	oui
amount	number > 0	oui
currency	CDF USD	oui
merchant_reference	string ≤128	oui (clé d'idempotency)
description	string ≤255	non
callback_url	string	non (surcharge le webhook de la clé)

GET /v1/payouts/{payout_id}

Statut d'un payout.

GET /v1/payouts

Lister vos payouts (status, recipient_id, page, page_size).

8.4 Balance

GET /v1/balance

Solde temps réel par (method, currency).

8.5 Cycle de vie d'un payout

Statut	Signification
RESERVED	Fonds bloqués, en attente d'envoi switch.
HOLD_REVIEW	Montant au-dessus du seuil, attente backoffice Moko.
DISPATCHED	Requête envoyée au switch.
WAITING_CALLBACK	Switch a accepté, attente confirmation opérateur.

COMPLETED	Bénéficiaire crédité. Le payout est terminé.
FAILED	Échec définitif. Fonds restitués.
EXPIRED	Pas de callback après 24h. Fonds restitués.
RELEASED	Annulé manuellement par backoffice Moko.

9. Les 4 identifiants dans votre code de production

La section 2 de ce guide a introduit les 4 identifiants. Cette section détaille leur usage concret dans votre code de production, avec les patterns recommandés.

9.1 Schéma de base recommandé

```
-- Table 1 : vos bénéficiaires
CREATE TABLE beneficiaries (
  id                BIGINT PRIMARY KEY,
  full_name         VARCHAR(255) NOT NULL,
  phone             VARCHAR(32) NOT NULL,
  operator           VARCHAR(16) NOT NULL,
  moko_recipient_id VARCHAR(64) NOT NULL UNIQUE, -- rec_...
  moko_status        VARCHAR(32) NOT NULL,       -- ACTIVE / PENDING_KYC / ...
  kyc_reference      VARCHAR(128),               -- votre référence KYC interne
  created_at         DATETIME NOT NULL,
  INDEX idx_moko_recipient (moko_recipient_id)
);

-- Table 2 : vos payouts
CREATE TABLE moko_payouts (
  id                BIGINT PRIMARY KEY,
  beneficiary_id     BIGINT NOT NULL,           -- FK vers beneficiaries
  merchant_reference VARCHAR(128) NOT NULL UNIQUE, -- clé d'idempotency
  moko_payout_id     VARCHAR(64),              -- PAY-... (rempli après réponse Moko)
  amount             DECIMAL(18,2) NOT NULL,
  currency           VARCHAR(8) NOT NULL,
  moko_status        VARCHAR(32) NOT NULL,     -- pending_send / RESERVED / COMPLETED
  ...
  business_context   VARCHAR(255),            -- ex. 'course_789'
  created_at         DATETIME NOT NULL,
  completed_at       DATETIME,
  INDEX idx_moko_payout (moko_payout_id),
  INDEX idx_status (moko_status),
  FOREIGN KEY (beneficiary_id) REFERENCES beneficiaries(id)
);
```

9.2 Flux de création d'un bénéficiaire

```
# 1. Construire merchant_recipient_id à partir de votre clé primaire :
beneficiary = Beneficiary.objects.get(pk=42)
merchant_recipient_id = f"teacher_{beneficiary.id}"
kyc_reference = f"TEACHER-CV-2026-{beneficiary.kyc_file_id:04d}"
```

```

# 2. Appeler Moko :
status, response = moko_client.create_recipient(
    merchant_recipient_id=merchant_recipient_id,
    full_name=beneficiary.full_name,
    phone=beneficiary.phone,
    operator=beneficiary.operator,
    kyc_reference=kyc_reference,
)

if status == 201:
    # 3. Stocker recipient_id :
    beneficiary.moko_recipient_id = response['recipient_id']
    beneficiary.moko_status = response['status']
    beneficiary.save()
elif status == 409: # déjà existant
    # Récupérer le recipient existant par GET
    ...
else:
    log.error("Moko create_recipient failed: %s %s", status, response)
    raise CreateRecipientFailed(response)

```

9.3 Flux de création d'un payout

```

# 1. Générer merchant_reference de manière déterministe :
course = Course.objects.get(pk=789)
teacher = course.teacher # le bénéficiaire
amount = course.compute_teacher_revenue() # 100 CDF
merchant_reference = f"course-{course.id}-teacher-{teacher.id}-payout"

# 2. Stocker en DB AVANT l'appel :
payout = MokoPayout.objects.create(
    beneficiary=teacher,
    merchant_reference=merchant_reference,
    amount=amount,
    currency='CDF',
    moko_status='pending_send',
    business_context=f"course_{course.id}",
)

# 3. Appeler Moko :
status, response = moko_client.create_payout(
    recipient_id=teacher.moko_recipient_id,
    amount=amount,
    currency='CDF',
    merchant_reference=merchant_reference,
    description=f"Revenus cours #{course.id}",
)

if status in (200, 202):
    # 4. Stocker payout_id :
    payout.moko_payout_id = response['payout_id']
    payout.moko_status = response['status'] # RESERVED ou HOLD_REVIEW

```

```

    payout.save()
else:
    payout.moko_status = 'send_failed'
    payout.save()
    log.error("create_payout failed: %s %s", status, response)
    raise CreatePayoutFailed(response)

```

9.4 Réception du webhook

```

# Endpoint POST /webhooks/moko-payout :
@app.route('/webhooks/moko-payout', methods=['POST'])
def moko_webhook():
    raw = request.get_data()
    sig = request.headers.get('X-Moko-Webhook-Signature', '')
    if not verify_webhook(raw, sig, WEBHOOK_SECRET):
        return Response(status=401)

    event = json.loads(raw)
    moko_payout_id = event['data']['payout_id'] # PAY-...
    new_status = event['data']['status']

    # Retrouver VOTRE payout par moko_payout_id :
    payout = MokoPayout.objects.filter(moko_payout_id=moko_payout_id).first()
    if not payout:
        # Cas rare : webhook pour un payout qu'on ne connaît pas.
        # Logger et répondre 200 quand même (idempotency côté Moko).
        log.warning("Unknown payout %s", moko_payout_id)
        return Response(status=200)

    # Mettre à jour le statut (idempotent) :
    if payout.moko_status != new_status:
        payout.moko_status = new_status
        if new_status == 'COMPLETED':
            payout.completed_at = datetime.utcnow()
            notify_teacher(payout) # envoyer SMS/email au bénéficiaire
            payout.save()

    return Response(status=200)

```

10. Webhooks — réception et vérification

Si vous configurez une `callback_url` sur votre clé API, Moko vous envoie un POST HTTPS à chaque transition d'état.

10.1 Événements

Événement	Quand
<code>payout.created</code>	Le payout vient d'être créé en <code>RESERVED</code> .
<code>payout.held_for_review</code>	Payout en <code>HOLD_REVIEW</code> .
<code>payout.processing</code>	Transition vers <code>DISPATCHED</code> / <code>WAITING_CALLBACK</code> .

payout.completed	Bénéficiaire crédité avec succès.
payout.failed	Échec définitif (FAILED, EXPIRED, RELEASED).

10.2 Format du payload

```
POST https://votre-domaine.com/votre-endpoint-webhook
Content-Type: application/json
X-Moko-Webhook-Signature: t=1717420800123,v1=64-char-hex
X-Moko-Webhook-Event: payout.completed
```

```
{
  "event": "payout.completed",
  "occurred_at": "2026-06-03T14:05:00Z",
  "data": {
    "payout_id": "PAY-...",
    "reference": "PAY-...",
    "recipient_id": "rec...",
    "amount": "100",
    "currency": "CDF",
    "method": "mpesa",
    "status": "COMPLETED",
    "switch_reference": "FP01...",
    "completed_at": "2026-06-03T14:05:00Z"
  }
}
```

10.3 Vérification de la signature

```
Signature attendue = HMAC-SHA256(webhook_secret, timestamp + "." + raw_body)

def verify_webhook(raw_body, header, webhook_secret):
    parts = dict(p.split("=", 1) for p in header.split(","))
    canonical = f"{parts['t']}".encode() + raw_body
    expected = hmac.new(webhook_secret, canonical, hashlib.sha256).hexdigest()
    return hmac.compare_digest(expected, parts.get("v1", ""))
```

10.4 Retries

- 1min → 5min → 30min → 2h → 12h → 24h.
- 6 tentatives au total avant abandon (~39h cumulées).
- Votre endpoint DOIT être idempotent : déduplication par payout_id.
- Vérifier la signature avant toute action métier, sinon n'importe qui peut vous envoyer de faux événements.

11. Gestion des erreurs et stratégie de retry

11.1 Format

```
{"detail": {"code": "limit_amount_per_day", "message": "..."}}
```

11.2 Codes HTTP

Code	Comportement attendu
200 / 201 / 202	Succès, continuer.
4xx (sauf 429)	NE PAS RETENTER. Corriger le payload.
429	Attendre puis retenter avec backoff.
5xx / timeout	Retenter avec backoff exponentiel (max 3 tentatives).

11.3 Codes métier fréquents

Code	Action
auth_signature_invalid	Vérifier clés, horloge, canonical (logger pour comparaison).
auth_nonce_replay	Générer un UUID v4 frais à chaque appel.
recipient_phone_invalid	Valider chez vous avant l'appel (format +243xxx).
recipient_cooldown	Patienter la fin du cooldown (24h).
recipient_already_exists	GET le recipient existant au lieu de POST.
insufficient_balance	Approvisionner Moko ou attendre 5 min (rebalancer auto).
limit_amount_per_*	Demander un relèvement du plafond au support.

11.4 Règle d'or pour les retries

À chaque retry, utilisez le MÊME merchant_reference. C'est ce qui garantit qu'il n'y a jamais de double prélèvement.

12. Idempotency en pratique

Rappel : merchant_reference est votre clé d'idempotency. Section 2.4 explique le concept. Cette section donne le pattern d'implémentation.

12.1 Pattern recommandé

```
def safe_create_payout(business_intent_id, beneficiary_id, amount, currency):
    # Étape 1 : récupérer ou créer la ligne en DB.
    payout, created = MokoPayout.objects.get_or_create(
        merchant_reference=f"intent-{business_intent_id}",
        defaults=dict(
            beneficiary_id=beneficiary_id,
            amount=amount, currency=currency,
            moko_status='pending_send',
        ),
```

```
)

# Étape 2 : si la réponse Moko est déjà connue, ne pas retenter.
if payout.moko_status not in ('pending_send', 'send_failed'):
    return payout

# Étape 3 : appeler Moko avec le merchant_reference déjà fixé.
status, resp = moko_client.create_payout(
    recipient_id=payout.beneficiary.moko_recipient_id,
    amount=payout.amount, currency=payout.currency,
    merchant_reference=payout.merchant_reference,
)

# Étape 4 : mettre à jour selon la réponse.
if status in (200, 202):
    payout.moko_payout_id = resp['payout_id']
    payout.moko_status = resp['status']
    payout.save()
return payout
```

13. Exemples bout-en-bout

13.1 Client Python minimal

```
import hashlib, hmac, json, secrets, time
from urllib.parse import urlencode, urlparse, parse_qs
import requests

BASE_URL = "https://payouts.gofreshpay.com"
PUBLIC_KEY = "pk_live..."
SECRET_KEY = b"sk_live..."

def _sign(method, path_with_query, body_bytes):
    parsed = urlparse(path_with_query)
    sorted_q = urlencode(sorted(parse_qs(parsed.query, keep_blank_values=True)))
    full_path = parsed.path + (f"?{sorted_q}" if sorted_q else "")
    body_hash = hashlib.sha256(body_bytes).hexdigest()
    ts = str(int(time.time() * 1000))
    nonce = secrets.token_hex(16)
    canonical = f"{method.upper()}\n{full_path}\n{body_hash}\n{ts}\n{nonce}".encode()
    sig = hmac.new(SECRET_KEY, canonical, hashlib.sha256).hexdigest()
    return {
        "X-Moko-Key": PUBLIC_KEY,
        "X-Moko-Timestamp": ts,
        "X-Moko-Nonce": nonce,
        "X-Moko-Signature": sig,
    }

def request(method, path, body=None):
    body_bytes = json.dumps(body, separators=(",", ":")).encode() if body else b""
    headers = _sign(method, path, body_bytes)
    headers["Content-Type"] = "application/json"
    r = requests.request(method, BASE_URL + path, data=body_bytes, headers=headers,
        timeout=30)
    return r.status_code, r.json() if r.content else None
```

13.2 Client Node.js minimal

```
const crypto = require('crypto');
const axios = require('axios');

const BASE_URL = 'https://payouts.gofreshpay.com';
const PUBLIC_KEY = process.env.MOKO_PUBLIC_KEY;
const SECRET_KEY = process.env.MOKO_SECRET_KEY;

function sign(method, path, query, bodyBytes) {
    const sortedQs = new URLSearchParams([...new URLSearchParams(query)].sort()).toString();
    const fullPath = sortedQs ? `${path}?${sortedQs}` : path;
    const bodyHash = crypto.createHash('sha256').update(bodyBytes).digest('hex');
    const ts = Date.now().toString();
    const nonce = crypto.randomUUID();
```

```
const canonical = [method.toUpperCase(), fullPath, bodyHash, ts, nonce].join('\n');
const sig = crypto.createHmac('sha256', SECRET_KEY).update(canonical).digest('hex');
return {'X-Moko-Key': PUBLIC_KEY, 'X-Moko-Timestamp': ts, 'X-Moko-Nonce': nonce, 'X-Moko-Signature': sig};
}
```

14. Checklist de mise en production

Sécurité

- Clés API dans un coffre-fort (pas dans le code).
- Aucun log de la secret_key ni du webhook_secret.
- HTTPS uniquement, certificat valide vérifié.
- IP allowlist activée sur la clé si IP de sortie fixe.
- Procédure documentée de révocation/rotation.

Robustesse

- Try/catch avec retry sur 5xx et timeout.
- merchant_reference généré et stocké AVANT l'appel.
- Horloge système synchronisée par NTP.
- Webhook endpoint idempotent (déduplication par payout_id).
- Webhook endpoint vérifie la signature avant toute action.
- Tests d'intégration sur les cas d'erreur.

Observabilité

- Logs d'audit côté votre serveur pour chaque payout.
- Métriques : taux de succès, latence p95, volume horaire.
- Alertes : taux d'échec > X%, HOLD_REVIEW > Y heures.

Procédures opérationnelles

- Contact Moko en cas d'incident.
- Notifications utilisateurs en cas d'échec.
- Réconciliation comptable périodique.

15. Support

- Email : henock.barakael@mokoafrika.com
- Console marchand : <https://cd.merchants.gofreshpay.com>
- Documentation OpenAPI : à la demande auprès du support.